Attention & Transformers

The simple version. One idea at a time. No coding required.

MATCH → **MIX** → **UPDATE**

Find useful information

Blend what you found

Build a better representation

30 minutes / beginner-friendly

Words need context

The same word can need different information.

river bank

We sat on the bank
beside the river.

money bank

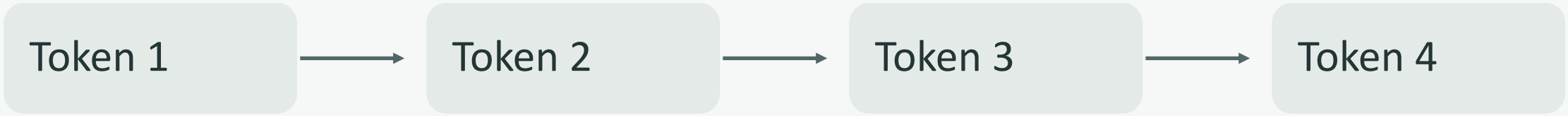
I deposited money
at the bank.

Same token, different context -> a different useful representation.

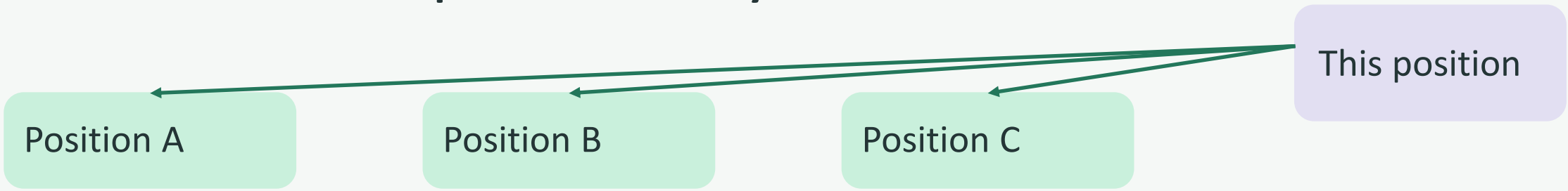
From carrying memory to reading context

LSTMs pass a state along. Attention creates direct information lookups.

LSTM: a chain of recurrent updates



Attention: read useful positions directly



Query, key, value: think of a library

An analogy for matching information, not a literal search engine.

Q / Query

The request

"Warm lunch"

K / Key

The matching features

"Soup, hot, lunch"

V / Value

The content to retrieve

The recipe information

Q + K decide how much. V supplies what gets contributed.

Attention is a weighted blend

Match the keys, turn scores into weights, then mix the values.



Illustrative weights only. They sum to 100%.

Attention output = weighted mixture of the value vectors.

The output is a vector, not a word

A tiny example: different values, different mixing amounts.

A: [10, 0]

80% -> [8, 0]

B: [0, 10]

10% -> [0, 1]

C: [5, 5]

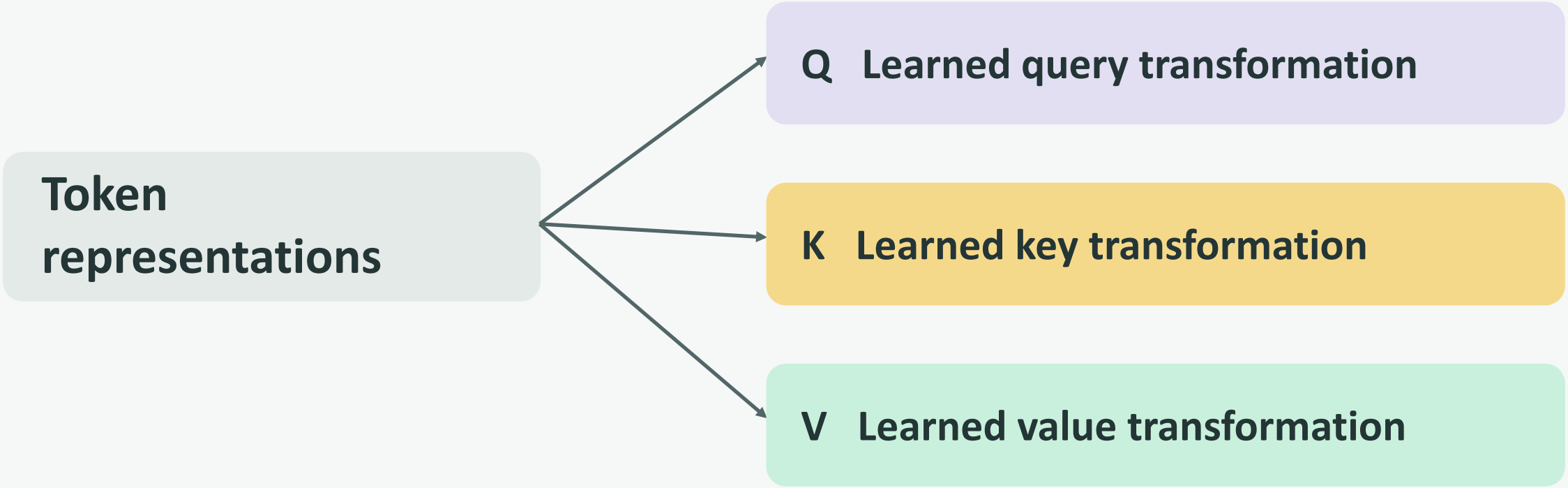
10% -> [0.5, 0.5]

Mix them -> [8.5, 1.5]

Not a new word. Not directly the next V. An input to the rest of the block.

Where do Q, K and V come from?

The model learns how to create them from token representations.



Training learns the transformations. The current input determines Q, K and V.

Self-attention: who is allowed to read whom?

Same mechanism, different access rules.

Encoder: both directions

	I	love	learning	today
I	read	read	read	read
love	read	read	read	read
learning	read	read	read	read
today	read	read	read	read

Causal decoder: no future

	I	love	learning	today
I	read	-	-	-
love	read	read	-	-
learning	read	read	read	-
today	read	read	read	read

Rows = positions asking. Columns = positions available to read.

At "love", a causal decoder reads "I love" to predict what comes next.

A Transformer is more than attention

These pieces work together to update token representations.

Position information

Make token order available.

Multi-head attention

Mix information in several learned ways.

Feed-forward network

Transform features at each position.

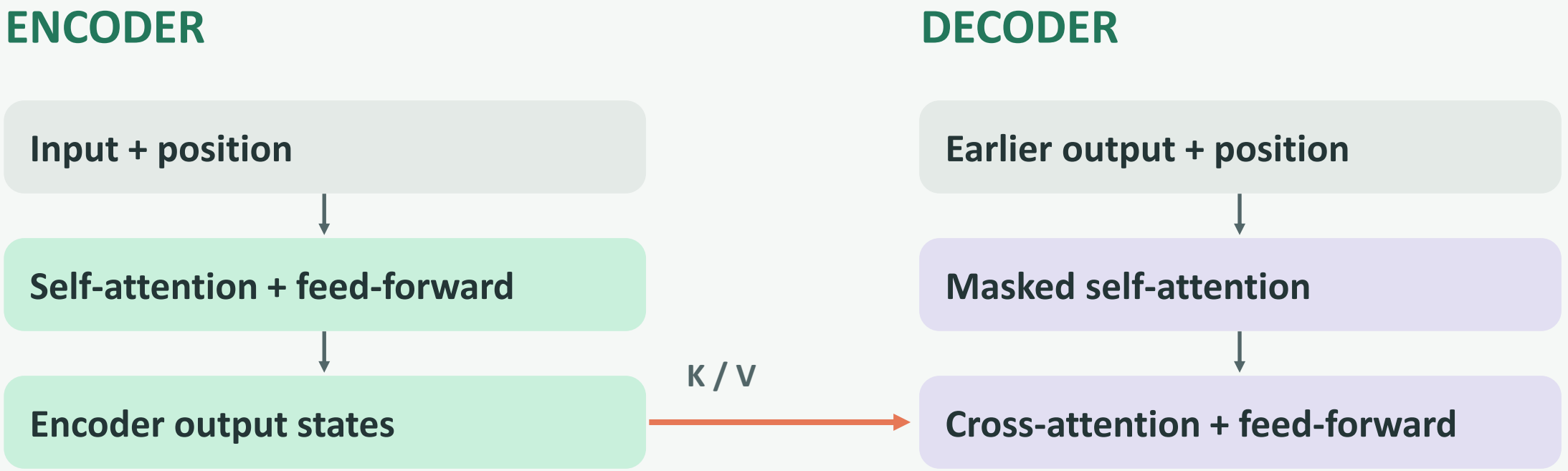
Residuals + normalization

Combine updates and regulate scales.

Repeat blocks -> richer contextual representations.

Encoder reads. Decoder generates.

A simple encoder-decoder Transformer; not every model needs both sides.



BERT-like: encoder only. GPT-like: decoder only, without cross-attention.

Generation: predict, append, repeat

The final representation becomes scores for possible next tokens.



Illustrative token probabilities, not measured model output.

Attention gathers information. The language-model head enables next-token prediction.

You only need three ideas

Questions welcome. No matrix derivations required.

01	**Match**	Queries match keys.
02	**Mix**	Weights blend values.
03	**Update**	A Transformer builds richer representations.

Check: What gets mixed? Why hide the future?